GUIA VISUAL 061

POSICIONAIS E GETOPTS

*A análise do script de coleta cresceu um `if` por vez: `if [ "$1" = "-v" ]; then … shift; fi`, três vezes. Funcionava — enquanto as opções viessem **na ordem dos `if`**, uma por vez, e nunca depois do arquivo. A análise de argumentos é a fronteira entre **o que o usuário digitou** e **o que o script executa**, e ela precisa de estrutura.*

061
GUIAS LINUX

CINCO INVOCAÇÕES RAZOÁVEIS — E NENHUMA MENSAGEM DE ERRO

`coletar -n -v dados`
O `-v` é testado **primeiro**: não é reconhecido, e `VERBOSE` fica 0.

`coletar -vn dados`
Agrupadas: `"-vn"` não é igual a `"-v"`. Nada é reconhecido.

`coletar dados -v`
Opção **depois** do operando: simplesmente ignorada.

`coletar -o dados`
`SAIDA` vira `dados` e o **arquivo fica vazio** — argumento faltando.

`coletar -- -rf`
O `--` vira **nome de arquivo**, e o `-rf` chega ao comando interno como **opção**.

GETOPTS CAMINHANDO PELO VETOR · `WHILE GETOPTS ':VNO:H' OPCAO`

`-vn` $1	`-o` $2	`saida.txt` $3	`alvo1` $4	`alvo2` $5

1ª	`opcao=v · OPTARG vazio · OPTIND=1`	ainda **dentro** do `-vn`: o *getopts* desagrupa sozinho.
2ª	`opcao=n · OPTARG vazio · OPTIND=2`	terminou o `-vn`; agora aponta para `$2`.
3ª	`opcao=o · OPTARG=saida.txt · OPTIND=4`	consumiu **dois** argumentos. Aceita colado também: `-osaida.txt`.
4ª	`devolve código ≠ 0 · OPTIND=4`	`$4` não começa com hífen: acabaram as opções, o laço termina.

`shift $(( OPTIND - 1 ))` — Descarta os três argumentos já analisados: agora `"$@"` contém **só os operandos** e `$#` vale 2. **Sem essa linha**, o laço seguinte processaria `-vn`, `-o` e `saida.txt` como alvos — o erro nº 1 com `getopts`, e ele não emite aviso nenhum: só um alvo chamado `-vn`.

o esqueleto que resolve os cinco casos

```
while getopts ':vno:h' opcao; do   # : inicial = modo silencioso
  case "$opcao" in                 # o : DEPOIS da letra = exige argumento
    v)  VERBOSE=1 ;;
    n)  SIMULAR=1 ;;
    o)  SAIDA="$OPTARG" ;;
    h)  uso; exit 0 ;;             # --help e uso bem-sucedido: stdout, codigo 0
    :)  erro "-$OPTARG exige um argumento"; exit 2 ;;
    \?) erro "opcao invalida: -$OPTARG"; uso >&2; exit 2 ;;
  esac
done
shift $(( OPTIND - 1 ))            # a linha que todo mundo esquece
(( $# )) || { uso >&2; exit 2; }   # erro de uso: stderr, codigo 2

# opcoes LONGAS: getopts nao faz. O laco com case faz -- e trata o --
while (( $# )); do case "$1" in
  -v|--verbose) VERBOSE=1; shift ;;
  -o|--saida)   SAIDA="$2"; shift 2 ;;
  --saida=*)    SAIDA="${1#*=}"; shift ;;
  --)           shift; break ;;    # fim das opcoes
  -*)           erro "opcao desconhecida: $1"; exit 2 ;;
  *)            break ;;           # primeiro operando
esac; done
```

O QUE COBRE	IF	WHILE+CASE	GETOPTS
Agrupadas `-vn`	não	não	sim
Colado `-osaida`	não	não	sim
Longas `--verbose`	não	sim	não
Ordem livre das opções	não	sim	sim
Separador `--`	não	sim	sim
Rejeita opção desconhecida	não	sim	sim

O `if` encadeado não tem caso de uso: custa o mesmo trabalho e falha em tudo. Curtas apenas → `getopts`; qualquer opção longa → laço com `case`. O `getopt` externo do `util-linux` faz as duas coisas, mas exige o idioma `eval set -- "$OPCOES"` e não existe no BSD/macOS.

Posicionais são um array: `"$@"` repassa, `"$*"` só exibe, e `${10}` exige as chaves — `$10` é lido como `$1` seguido de `0`. `set -- a b c` reescreve a lista (o `--` inicial protege argumento com hífen). E `shift` **falha com código 1** se pedir mais do que existe, sem alterar nada: teste `$#` antes, ou o laço nunca termina.

Todo caminho vindo de variável entra no comando depois de um `--`. Se `$arquivo` valer `-rf` ou `--no-preserve-root`, o utilitário lê o nome como **opção**: escreva `rm -- "$arquivo"`, `cp -- "$origem" "$destino"`, `chown -- "$dono" "$alvo"`. Vale